

UVM Verification of a Synchronous FIFO

Prepared by:

Arwa Ashraf Kantoush

1. Introduction and design under test

This report documents a complete UVM verification environment for a synchronous FIFO. The environment drives constrained-random and directed stimulus through five UVM sequences, checks every DUT output against a reference model in the scoreboard, collects functional coverage, and uses bound SystemVerilog assertions to check the flags, the registered outputs and the internal pointers. Six RTL bugs were found and fixed; section 7 shows the code before and after each fix.

1.1 Design parameters and ports

Signal	Direction	Description
clk	input	Clock, generated in the top module
rst_n	input	Active-low asynchronous reset
wr_en, rd_en	input	Write and read requests
data_in [15:0]	input	Write data (FIFO_WIDTH = 16)
data_out [15:0]	output	Registered read data. It is not reset; it holds the last value read
wr_ack	output	Registered; high one cycle after an accepted write
overflow	output	Registered; high one cycle after a write to a full FIFO
underflow	output	Registered; high one cycle after a read from an empty FIFO
full, empty	output	Combinational from count (count == 8, count == 0)
almostfull, almostempty	output	Combinational from count (count == 7, count == 1)

The FIFO depth is 8. The internal state is a memory array, a write pointer, a read pointer and a count register. A write is accepted when count is below the depth and a read is accepted when count is not zero. When a read and a write arrive together, count goes up by one if the FIFO is empty (only the write happens), goes down by one if it is full (only the read happens), and stays the same otherwise.

2. Verification plan

Each requirement has a label, the stimulus that exercises it, the functional coverage that shows it was exercised, and the check that proves it behaved correctly. A dash in the coverage column means the item is covered by checks only.

Label	Design requirement	Stimulus generation	Functional coverage	Functionality check
FIFO_1	When reset is asserted, the write pointer, read pointer and count are cleared, so the FIFO is empty	FIFO_reset_seq at the start; random resets (rst_n low about 3% of the time) in every sequence	-	rst_n_propert assertion; the scoreboard reference model clears its pointers and count
FIFO_2	A write to a non-full FIFO stores the data, increments count and raises wr_ack in the next cycle	FIFO_main_write_only _seq; also the random sequence	cp_wr_en, cp_wr_ack, cx_wr_rd_wr_ack	wr_ack_propert assertion; scoreboard compares wr_ack
FIFO_3	A write to a full FIFO is ignored and overflow is raised in the next cycle	FIFO_main_write_only _seq (many more writes than the depth); also the random sequence	cp_overflow, cx_wr_rd_overflow	overflow_propert assertion; scoreboard compares overflow
FIFO_4	A read from a non-empty FIFO returns the oldest word on the next clock edge and decrements count	FIFO_main_read_only_ seq after the FIFO is filled; also the write-read and random sequences	cp_rd_en, cx_wr_rd_empty	Scoreboard compares data_out with the reference model
FIFO_5	A read from an empty FIFO is ignored and underflow is raised in the next cycle	FIFO_main_read_only_ seq (more reads than stored words); also the random sequence	cp_underflow, cx_wr_rd_underflo w	underflow_propert assertion; scoreboard compares underflow
FIFO_6	A simultaneous read and write, FIFO neither full nor empty: both complete and count is unchanged	FIFO_main_write_read _seq; also the random sequence	cx_wr_rd_almoste mpty, cx_wr_rd_almostful l	Scoreboard (data_out and flags); Pointer_threshold_ propert

FIFO_7	A simultaneous read and write on an empty FIFO: the write is accepted, the read raises underflow and count increments	FIFO_main_write_read_seq (starts from the FIFO drained by the read-only sequence); also the random sequence	cx_wr_rd_empty, cx_wr_rd_underflow	wr_ack_propert and underflow_propert; scoreboard
FIFO_8	A simultaneous read and write on a full FIFO: the read is accepted, the write raises overflow and count decrements	FIFO_main_rand_seq (random enables with write-heavy traffic)	cx_wr_rd_full, cx_wr_rd_overflow	overflow_propert; scoreboard compares data_out and the flags
FIFO_9	full is asserted exactly when count equals the depth	FIFO_main_write_only_seq; also the random sequence	cp_full_pre, cx_wr_rd_full	a_full assertion (continuous, both directions)
FIFO_10	empty is asserted exactly when count is 0	FIFO_reset_seq, FIFO_main_read_only_seq; also the random sequence	cp_empty_pre, cx_wr_rd_empty	a_empty assertion
FIFO_11	almostfull is asserted exactly when count is depth - 1	FIFO_main_write_only_seq; also the random sequence	cp_almostfull, cx_wr_rd_almostfull	a_afull assertion
FIFO_12	almostempty is asserted exactly when count is 1	FIFO_main_read_only_seq, FIFO_main_write_read_seq; also the random sequence	cp_almostempty, cx_wr_rd_almostempty	a_aempty assertion
FIFO_13	The write and read pointers wrap to 0 after the last location	Write-only, read-only and random sequences (long traffic through all locations)	-	wr_ptr_wrap_propert and rd_ptr_wrap_propert
FIFO_14	The pointers and count always stay in their legal range	FIFO_main_rand_seq (random traffic with random resets)	-	Pointer_threshold_propert (checked every clock edge)
FIFO_15	Data is read out in the order it was written with no corruption, including after wraparound and reset	Random data_in in every sequence	-	Scoreboard reference model compares data_out on every item

3. UVM testbench structure and operation

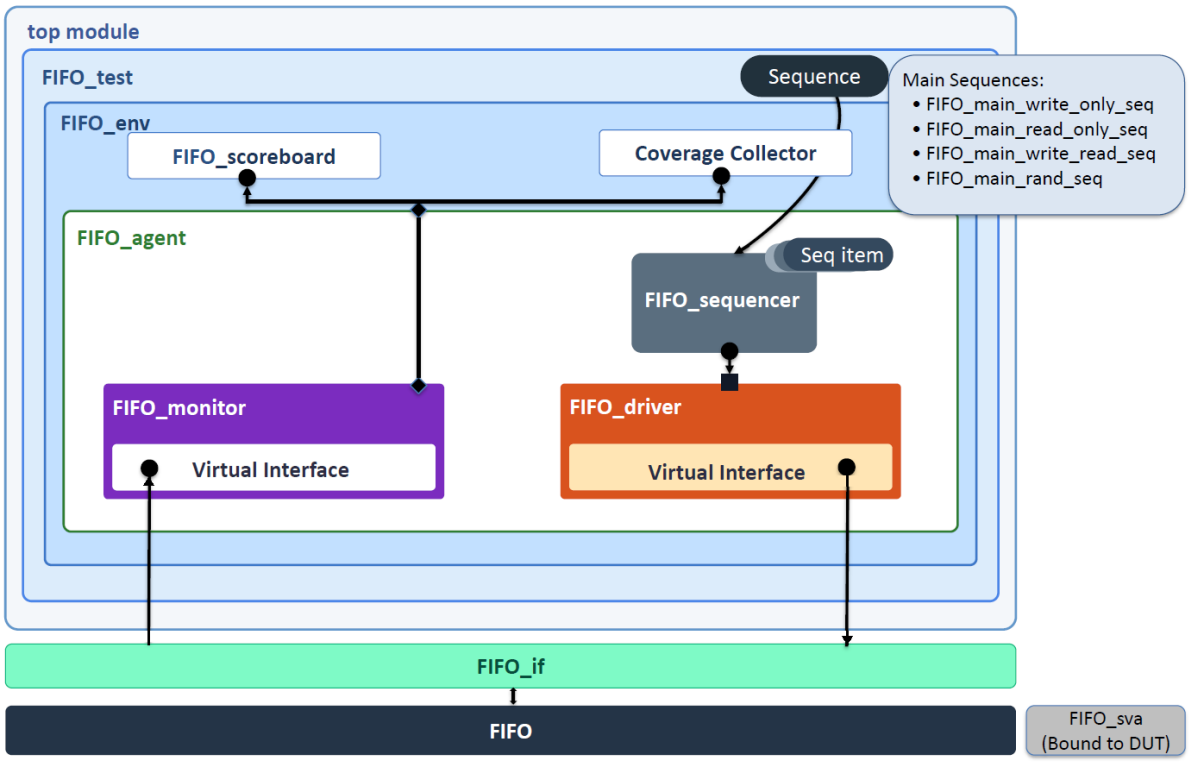


Figure 1: UVM testbench structure (FIFO_test also starts FIFO_reset_seq before the four main sequences listed)

3.1 How the testbench works

Top module: The top module generates the clock, instantiates the interface FIFO_if and the FIFO DUT, and binds the assertion module FIFO_sva into the DUT, which gives the assertions access to the internal signals count, wr_ptr and rd_ptr. It then stores the virtual interface in uvm_config_db and calls run_test.

Test (FIFO_test): The test builds the environment and the configuration object FIFO_config, gets the virtual interface from uvm_config_db, stores it in the configuration object and publishes that object so the driver and monitor can read it. In the run phase it raises an objection and starts the sequences on the agent sequencer one after the other: reset, write only, read only, write and read together, then the random sequence. It drops the objection when the last sequence ends.

Sequence item (FIFO_seq_item): The item holds the randomized inputs (rst_n, wr_en, rd_en, data_in) and the observed outputs (data_out, wr_ack, overflow, underflow and the four flags). Its constraints keep reset rare and bias the enables, write high 70% of the time and read high 30%, so that the FIFO fills up and the overflow cases are reached. The directed sequences override the enables with inline constraints on randomize().

Driving: The sequencer hands each item to FIFO_driver through get_next_item. The driver drives only the DUT inputs (rst_n, wr_en, rd_en, data_in) onto the virtual interface, waits for the next falling clock edge and calls item_done.

Monitoring: FIFO_monitor waits for a rising clock edge and then a 1 ns delay, and only then samples every interface signal, inputs and outputs, into a new sequence item that it writes to its analysis port. The clock period is 10 ns and the driver changes the inputs only at the falling edge, so at the sampling point the DUT has already responded to the current inputs and the driver has not yet applied the next ones. Sampling at this fixed point removes the race that would exist if the monitor and the driver both reacted to the same falling edge, where the simulator could let the monitor see the next transaction's inputs next to the current outputs.

Analysis: The monitor's analysis port reaches both the scoreboard and the coverage collector. FIFO_scoreboard contains a cycle-accurate reference model of the FIFO (memory, pointers and count). For every item it updates the model and compares the DUT's data_out, wr_ack, overflow, underflow, full, empty, almostfull and almostempty with the expected values using the case-equality operator, then counts correct and failed transactions and prints both in the report phase. FIFO_coverage samples the covergroup on every item.

Assertions: FIFO_sva is bound inside the DUT. Because full, empty, almostfull and almostempty are combinational and follow count with no clock in between, they are checked in an always_comb block with deferred final assertions, which look at them continuously and compare in both directions. The registered signals (wr_ack, overflow, underflow) and the pointers are checked with clocked properties on the rising edge using the non-overlapped implication. Each clocked assertion has a matching cover property.

4. UVM sequences and waveform snippets

Five sequences are started by FIFO_test in the order below. Each snippet is a QuestaSim waveform of the interface signals.

4.1 FIFO_reset_seq

Three items with rst_n = 0, wr_en = 0, rd_en = 0 and data_in = 0, run at the start of the simulation. Holding reset for several clock edges guarantees that the monitor samples it and that the reference model starts from the reset state. The driver applies the reset to the interface, which clears the pointers, the count and the registered outputs wr_ack, overflow and underflow, so empty is high and full is low.

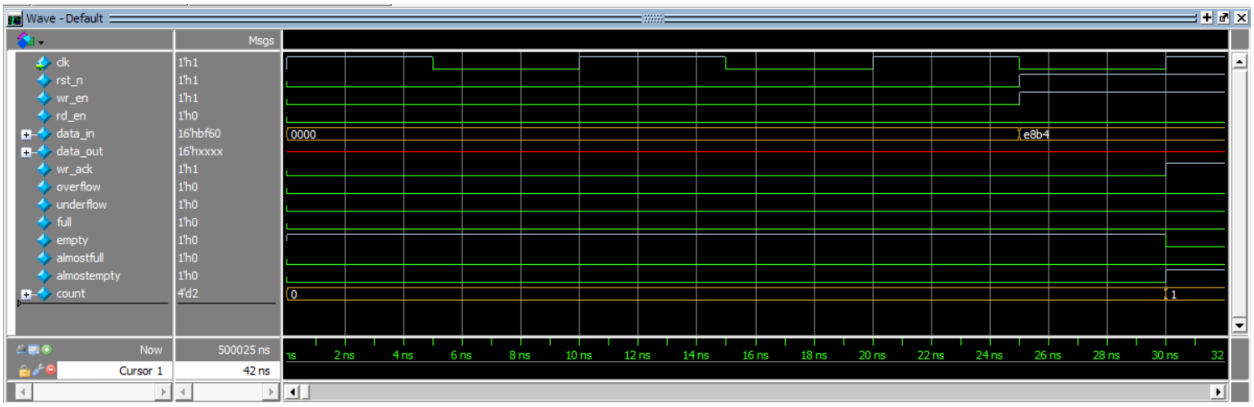


Figure 2: reset sequence

4.2 FIFO_main_write_only_seq

Ten thousand items with wr_en = 1 and rd_en = 0. rst_n is not forced, so it is low in about 3% of the items and the FIFO is reset about every 33 items; the short reset visible in the waveform at about 85 ns is one of these random resets. Between resets the FIFO fills to its depth of 8 and every further write is rejected. In the waveform wr_ack goes high one cycle after each accepted write, empty falls, almostfull is high at seven stored words and full goes high at eight. After that wr_ack drops and overflow rises for every rejected write.

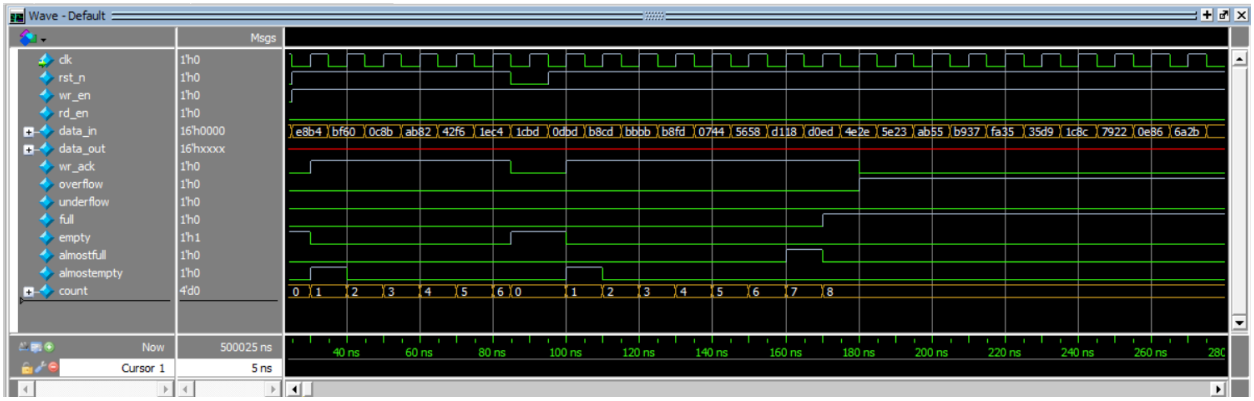


Figure 3: write-only sequence (a random reset is visible at about 85 ns)

4.3 FIFO_main_read_only_seq

Ten thousand items with wr_en = 0 and rd_en = 1. rst_n is not forced, so random resets (about 3% of the items) also occur here. The sequence drains the words stored by the previous sequence (8 in the waveform below) and then keeps reading from the empty FIFO. data_out returns the words in the order they were written, almostempty is high at the last stored word, then empty rises and underflow is raised one cycle after each read that finds the FIFO empty.

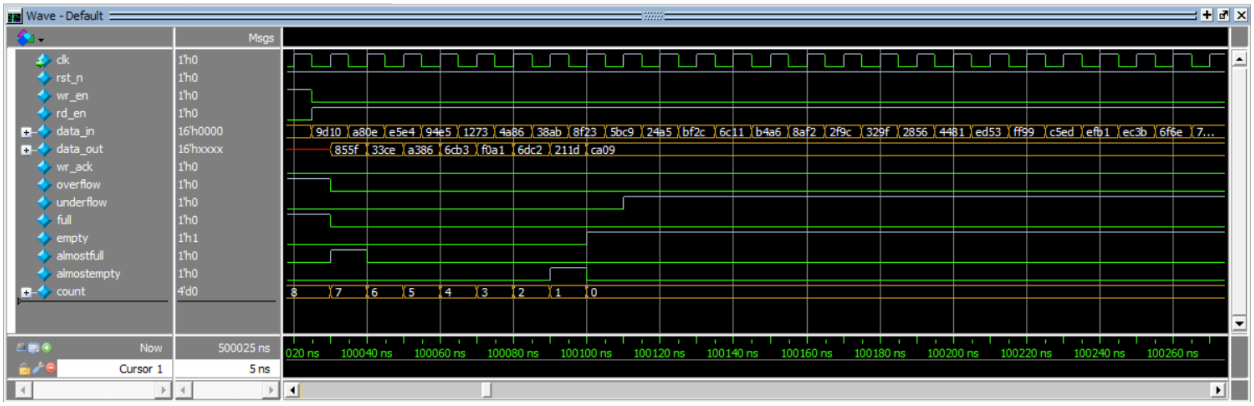


Figure 4: read-only sequence

4.4 FIFO_main_write_read_seq

Ten thousand items with wr_en = 1 and rd_en = 1. rst_n is not forced, so random resets (about 3% of the items) also occur, as the resets in the waveform show. The sequence starts from an empty FIFO. In the first cycle the write is accepted and the read is ignored (underflow is raised), so count becomes 1. From then on every cycle stores one word and reads one word, count holds at one entry until the next reset (the first cycle after each reset repeats the underflow pulse), almostempty stays high and data_out follows data_in with a one-item delay. This sequence covers the simultaneous read and write case at the empty boundary.

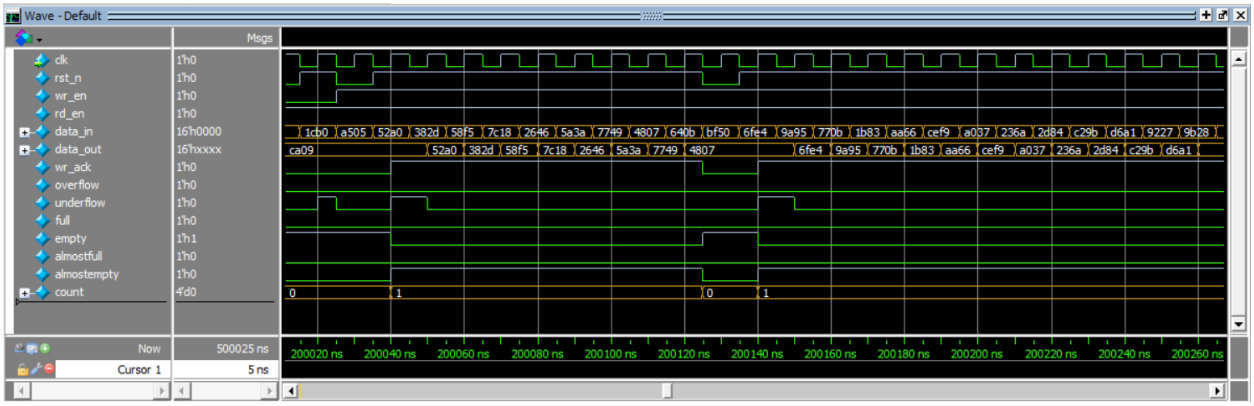


Figure 5: write-and-read sequence (random resets are visible in the waveform)

4.5 FIFO_main_rand_seq

Twenty thousand fully randomized items that use the constraints of the sequence item: rst_n low a few percent of the time, wr_en high 70% and rd_en high 30%. This sequence provides the long mixed traffic that reaches every flag, overflow and underflow, simultaneous read and write at the full boundary and in the middle of the FIFO, pointer wraparound and random resets. It is the only sequence that exercises a read and a write together on a full FIFO.

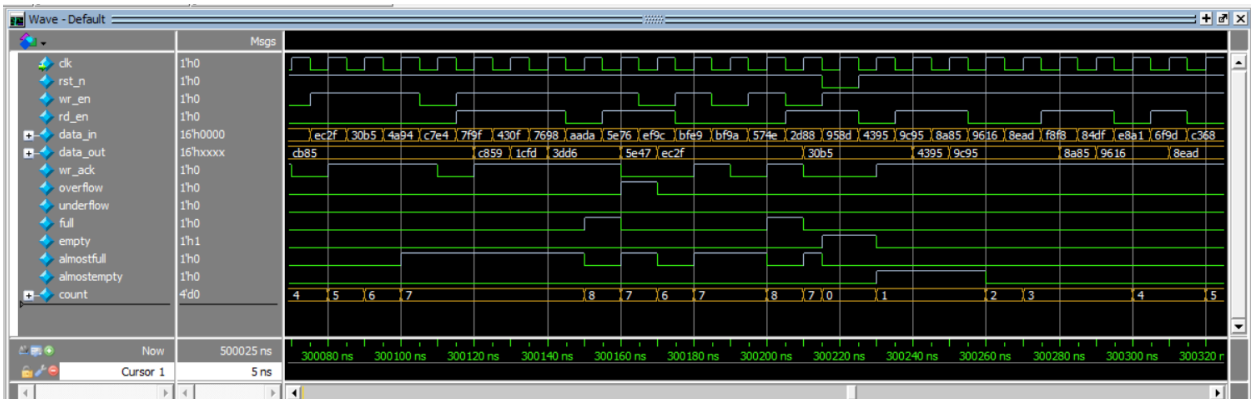


Figure 6: random sequence

5. Assertions

The assertions are in FIFO_sva and are bound to the DUT in the top module. The first four are combinational checks written in an always_comb block with deferred final assertions, so they run continuously. All the others are clocked on the rising edge.

Feature	Assertion	Name
full is asserted exactly when count equals the depth	`assert final (full == (count == FIFO_DEPTH))`	a_full
empty is asserted exactly when count is 0	`assert final (empty == (count == 0))`	a_empty
almostfull is asserted exactly when count is depth - 1	`assert final (almostfull == (count == FIFO_DEPTH-1))`	a_afull
almostempty is asserted exactly when count is 1	`assert final (almostempty == (count == 1))`	a_aempty
Reset clears the write pointer, read pointer and count	`@(posedge clk) (rst_n == 0) |-> (count == 0 && rd_ptr == 0 && wr_ptr == 0)`	rst_n_propert
A write to a non-full FIFO raises wr_ack in the next cycle	`@(posedge clk) disable iff (!rst_n) (wr_en && count < FIFO_DEPTH) |=> wr_ack`	wr_ack_propert
A write to a full FIFO raises overflow in the next cycle	`@(posedge clk) disable iff (!rst_n) (full & wr_en) |=> overflow`	overflow_propert
A read from an empty FIFO raises underflow in the next cycle	`@(posedge clk) disable iff (!rst_n) (rd_en && count == 0) |=> underflow`	underflow_propert
The write pointer wraps to 0 after the last location	`@(posedge clk) disable iff (!rst_n) (wr_ptr == FIFO_DEPTH-1 && wr_en && !full) |=> (wr_ptr == 0)`	wr_ptr_wrap_propert
The read pointer wraps to 0 after the last location	`@(posedge clk) disable iff (!rst_n) (rd_ptr == FIFO_DEPTH-1 && rd_en && !empty) |=> (rd_ptr == 0)`	rd_ptr_wrap_propert

Feature	Assertion	Name
Pointers and count always stay in their legal range	`@(posedge clk) disable iff (!rst_n) (rd_ptr < FIFO_DEPTH && wr_ptr < FIFO_DEPTH && count <= FIFO_DEPTH)`	Pointer_threshold_propert

The flag checks use equality instead of a one-way implication, so they also catch a flag that is high when count says it should be low. A deferred final assertion is evaluated at the end of the time step, so the small delay between count changing and the flag following it does not produce false failures.

6. Coverage reports

6.1 Code coverage

Code coverage was collected on the DUT (design unit work.FIFO) with branch, condition, expression, statement, FSM and toggle coverage enabled (+cover=bcesft). The full report is in coverage_rpt.txt.

Coverage type	Bins	Hits	Misses	Coverage
Branch	26	26	0	100.00%
Condition	16	16	0	100.00%
Statement	29	29	0	100.00%
Toggle	106	106	0	100.00%

Every branch, condition and statement of the RTL executed, and every signal toggled in both directions. This includes the branch where a read and a write arrive together on a full FIFO, which only the random sequence reaches. The top module also shows 100% statement and toggle coverage.

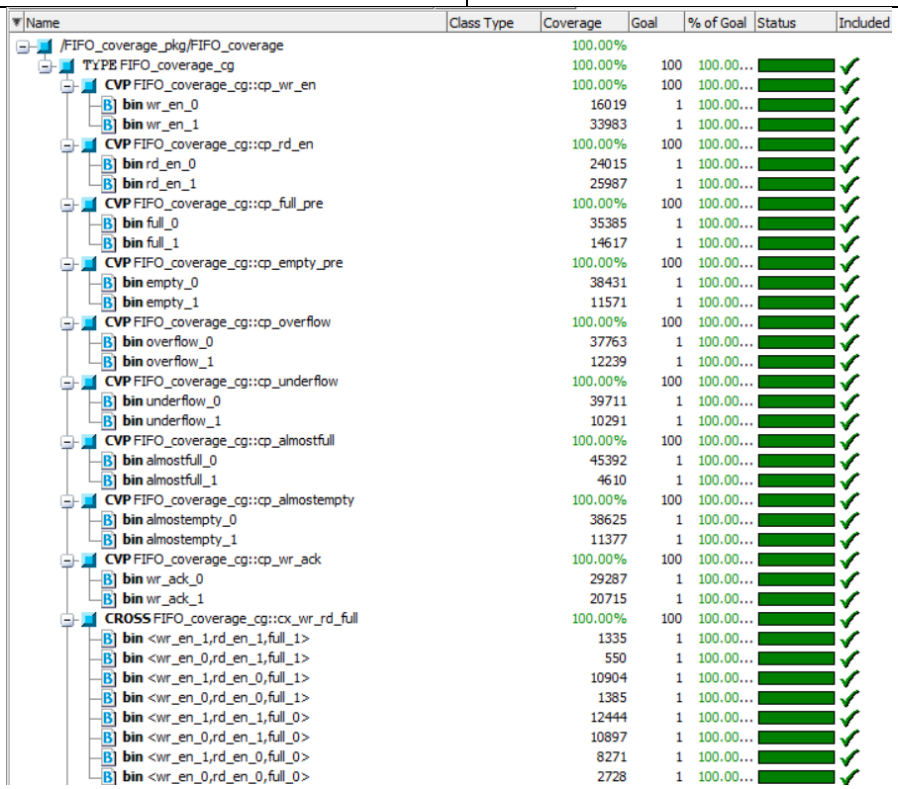
6.2 Functional coverage

The covergroup in FIFO_coverage has 9 coverpoints and 7 crosses, 68 bins in total. All 68 bins were hit, giving 100.00% (see coverage_rpt.txt). The coverpoints cp_wr_en and cp_rd_en sample the request inputs, and cp_overflow, cp_underflow, cp_almostfull, cp_almostempty and cp_wr_ack sample the outputs after the clock edge. The coverpoints cp_full_pre and cp_empty_pre sample the state of the FIFO before the transaction, taken from the previous item.

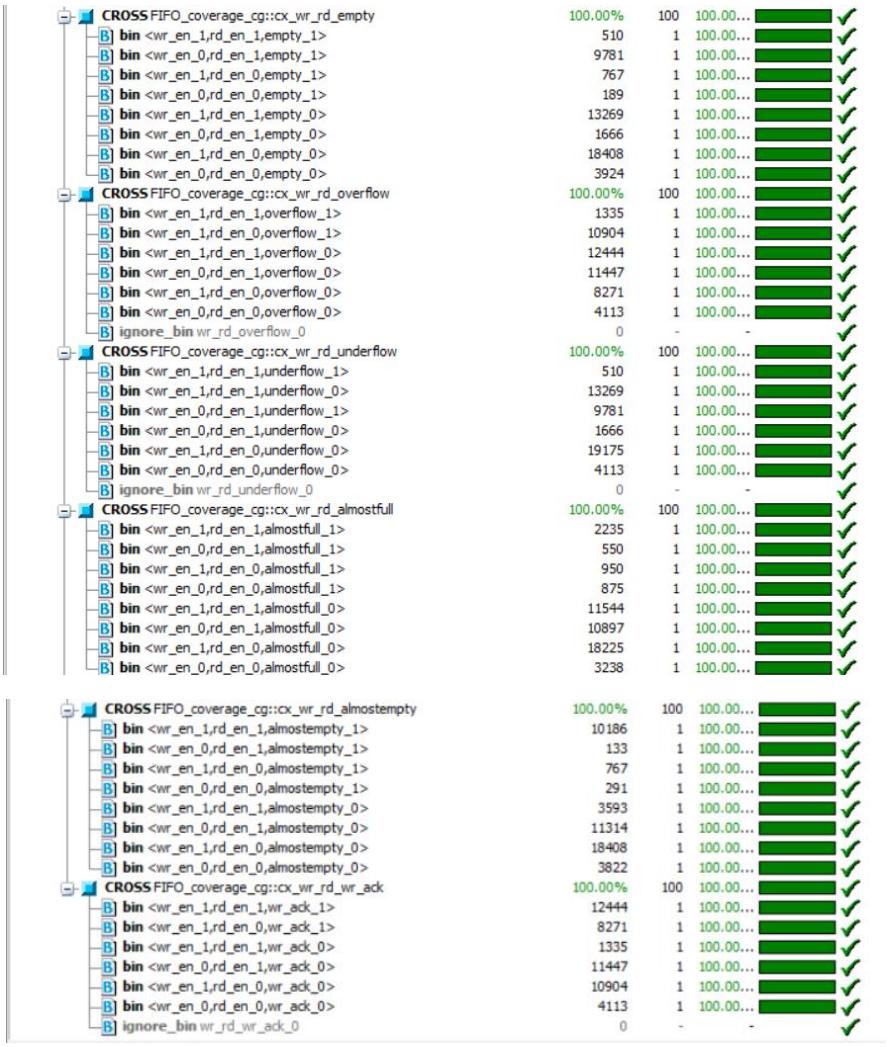
Coverpoint	What it samples	Bins
cp_wr_en, cp_rd_en	The request inputs	0 and 1 each
cp_full_pre, cp_empty_pre	full and empty before the transaction	0 and 1 each
cp_wr_ack, cp_overflow, cp_underflow	The registered outputs	0 and 1 each
cp_almostfull, cp_almostempty	The almost flags	0 and 1 each

Cross	Bins
cx_wr_rd_full (wr_en, rd_en, full before the transaction)	8
cx_wr_rd_empty (wr_en, rd_en, empty before the transaction)	8
cx_wr_rd_overflow	6 (2 ignored: overflow cannot be high without a write)

Cross	Bins
cx_wr_rd_underflow	6 (2 ignored: underflow cannot be high without a read)
cx_wr_rd_almostfull	8
cx_wr_rd_almostempty	8
cx_wr_rd_wr_ack	6 (2 ignored: wr_ack cannot be high without a write)



Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/FIFO_coverage_pkg/FIFO_coverage		100.00%				
TYPE FIFO_coverage_cg		100.00%	100	100.00...		
CVP FIFO_coverage_cg::cp_wr_en		100.00%	100	100.00...		
bin wr_en_0		16019	1	100.00...		
bin wr_en_1		33983	1	100.00...		
CVP FIFO_coverage_cg::cp_rd_en		100.00%	100	100.00...		
bin rd_en_0		24015	1	100.00...		
bin rd_en_1		25987	1	100.00...		
CVP FIFO_coverage_cg::cp_full_pre		100.00%	100	100.00...		
bin full_0		35385	1	100.00...		
bin full_1		14617	1	100.00...		
CVP FIFO_coverage_cg::cp_empty_pre		100.00%	100	100.00...		
bin empty_0		38431	1	100.00...		
bin empty_1		11571	1	100.00...		
CVP FIFO_coverage_cg::cp_overflow		100.00%	100	100.00...		
bin overflow_0		37763	1	100.00...		
bin overflow_1		12239	1	100.00...		
CVP FIFO_coverage_cg::cp_underflow		100.00%	100	100.00...		
bin underflow_0		39711	1	100.00...		
bin underflow_1		10291	1	100.00...		
CVP FIFO_coverage_cg::cp_almostfull		100.00%	100	100.00...		
bin almostfull_0		45392	1	100.00...		
bin almostfull_1		4610	1	100.00...		
CVP FIFO_coverage_cg::cp_almostempty		100.00%	100	100.00...		
bin almostempty_0		38625	1	100.00...		
bin almostempty_1		11377	1	100.00...		
CVP FIFO_coverage_cg::cp_wr_ack		100.00%	100	100.00...		
bin wr_ack_0		29287	1	100.00...		
bin wr_ack_1		20715	1	100.00...		
CROSS FIFO_coverage_cg::cx_wr_rd_full		100.00%	100	100.00...		
bin <wr_en_1,rd_en_1,full_1>		1335	1	100.00...		
bin <wr_en_0,rd_en_1,full_1>		550	1	100.00...		
bin <wr_en_1,rd_en_0,full_1>		10904	1	100.00...		
bin <wr_en_0,rd_en_0,full_1>		1385	1	100.00...		
bin <wr_en_1,rd_en_1,full_0>		12444	1	100.00...		
bin <wr_en_0,rd_en_1,full_0>		10897	1	100.00...		
bin <wr_en_1,rd_en_0,full_0>		8271	1	100.00...		
bin <wr_en_0,rd_en_0,full_0>		2728	1	100.00...		



QuestaSim Covergroups window: all coverpoints and crosses at 100%

6.3 Sequential domain coverage

Sequential domain coverage shows whether the temporal scenarios described by the assertions really happened. Each assertion has a cover property, and all of them were hit. The table compares FIFO_sva before and after the RTL fixes. In the run against the original design a_afull failed 2469 times and underflow_propert failed 278 times (Figure 7); every other assertion passed.

Run	Assertions	Failing assertions	Assertions without failures	Assertion coverage
Before RTL fixes (FIFO_before.v)	11	2	9 of 11	81.81%
After RTL fixes (FIFO.sv)	11	0	11 of 11	100.00%

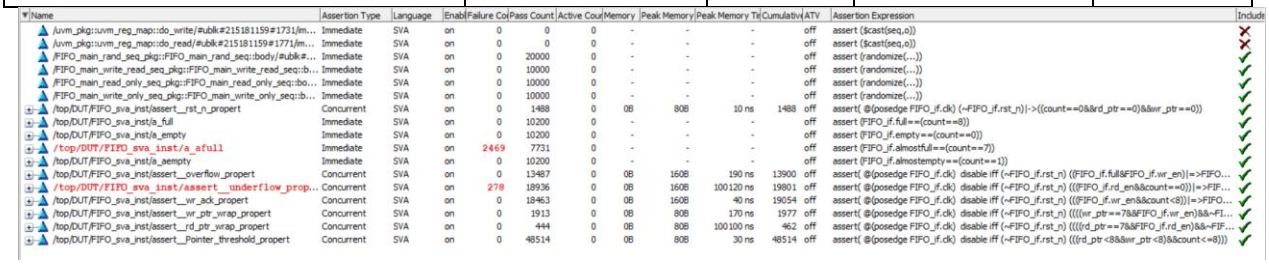


Figure 7: assertions window before the fixes

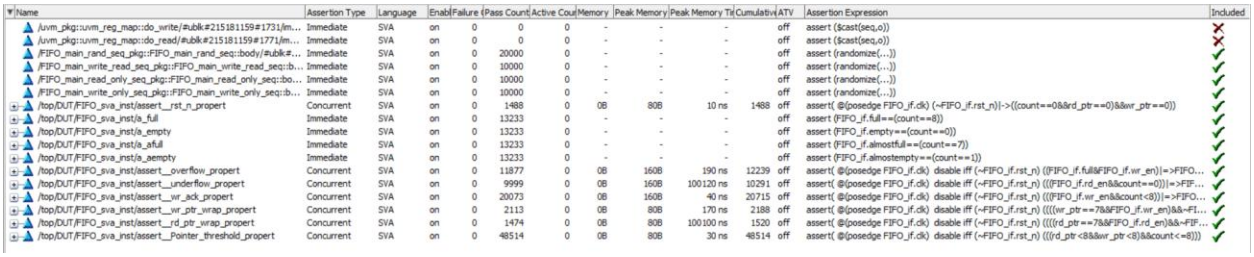


Figure 8: assertions window after the fixes

6.4 Scoreboard result

After the fixes the scoreboard reports Total failed transactions: 0 and the UVM report summary shows 0 UVM_ERROR and 0 UVM_FATAL. The sequences drive 50003 items in total: 3 reset, 10000 write-only, 10000 read-only, 10000 write-and-read and 20000 random. The scoreboard processed 50002 of them, because the first reset item is applied before the first sampled clock edge.

```
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test FIFO_test...
# UVM_INFO FIFO_test.sv(50) @ 0: uvm_test_top [run_phase] Reset Asserted
# ****************************************************************************
# * Questa UVM Transaction Recording Turned ON.                              *
# * recording_detail has been set.                                           *
# *  To turn off, set 'recording_detail' to off:                             *
# * uvm_config_db#(int)            ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# ****************************************************************************
# UVM_INFO FIFO_test.sv(52) @ 25: uvm_test_top [run_phase] Reset Deasserted
# UVM_INFO FIFO_test.sv(55) @ 25: uvm_test_top [run_phase] Stimulus Generation Started
# UVM_INFO FIFO_test.sv(57) @ 100025: uvm_test_top [run_phase] Stimulus Generation Ended
# UVM_INFO FIFO_test.sv(60) @ 100025: uvm_test_top [run_phase] Stimulus Generation Started
# UVM_INFO FIFO_test.sv(62) @ 200025: uvm_test_top [run_phase] Stimulus Generation Ended
# UVM_INFO FIFO_test.sv(65) @ 200025: uvm_test_top [run_phase] Stimulus Generation Started
# UVM_INFO FIFO_test.sv(67) @ 300025: uvm_test_top [run_phase] Stimulus Generation Ended
# UVM_INFO FIFO_test.sv(70) @ 300025: uvm_test_top [run_phase] Stimulus Generation Started
# UVM_INFO FIFO_test.sv(72) @ 500025: uvm_test_top [run_phase] Stimulus Generation Ended
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 500025: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO FIFO_scoreboard.sv(126) @ 500025: uvm_test_top.env.sb [report_phase] Total successful transactions: 50002
# UVM_INFO FIFO_scoreboard.sv(127) @ 500025: uvm_test_top.env.sb [report_phase] Total failed transactions: 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :   16
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [report_phase]     2
# [run_phase]    10
# ** Note: $finish    : D:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 500025 ns  Iteration: 61  Instance: /top
# Break in Task uvm_pkg/uvm_root::run_test at D:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
# QuestaSim-64 vcover 2021.1 Coverage Utility 2021.01 Jan 19 2021
# Start time: 16:26:02 on Oct 23,2026
# vcover report FIFO_cov.ucdb -details -annotate -all -output coverage_rpt.txt
# End time: 16:26:02 on Oct 23,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

Figure 9: transcript summary

7. Bug report

Six bugs were found in the original FIFO. The testbench was run once against the original design (FIFO_before.v) and once against the fixed design (FIFO.sv). The table lists each bug and how it was detected; the entries below show the code before and after each fix. Against the original design the scoreboard reported 24608 failed transactions out of 50002 (Figure 10).

#	Bug	Effect	Detected by
1	wr_ack was not reset	wr_ack unknown or stale after reset	Waveform: wr_ack stays 1 while rst_n = 0 (evidence below)
2	overflow was not reset	overflow unknown or stale after reset	Waveform and scoreboard: overflow stays 1 while rst_n = 0 (scoreboard error at 500021 ns)
3	overflow was not cleared by an accepted write	overflow stayed high after a later accepted write	Waveform: overflow stays 1 while wr_ack = 1 and the FIFO is not full
4	underflow was a combinational assign, not a registered output	underflow followed rd_en and empty in the same cycle instead of being held one cycle	underflow_propert assertion (278 failures)
5	count was not updated when wr_en and rd_en were both high	Wrong count, so full, empty and the almost flags were wrong afterwards	Waveform: empty and underflow stay 1 while wr_en = rd_en = 1
6	almostfull compared count with depth - 2	almostfull was high at 6 stored words instead of 7	a_afull assertion (2469 failures); the scoreboard shows DUT almostfull = 0 while the reference is 1

```
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :   16
# UVM_WARNING :    0
# UVM_ERROR :24608
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [report_phase]     2
# [run_phase] 24618
# ** Note: $finish    : D:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 500025 ns  Iteration: 61  Instance: /top
# 1
# Break in Task uvm_pkg/uvm_root::run_test at D:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
# QuestaSim-64 vcover 2021.1 Coverage Utility 2021.01 Jan 19 2021
# Start time: 19:01:08 on Oct 24,2026
# vcover report FIFO_cov.ucdb -details -annotate -all -output coverage_rpt.txt
# End time: 19:01:08 on Oct 24,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

Figure 10: scoreboard result against FIFO_before.v (24618 failed transactions)

Bug 1: wr_ack was not reset

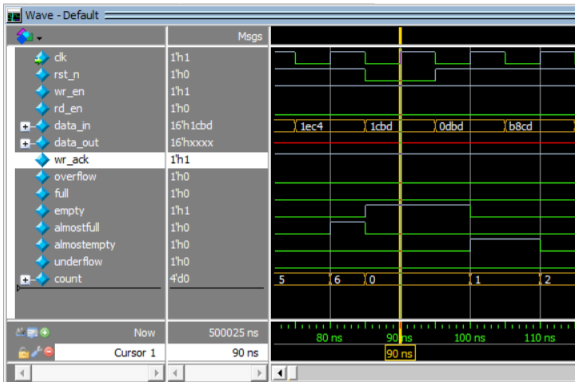
Before:

```
if (!rst_n) begin
    wr_ptr <= 0;
end
```

After:

```
if (!rst_n) begin
    wr_ptr <= 0;
    wr_ack <= 0;
    overflow <= 0;   // also fixes bug 2
end
```

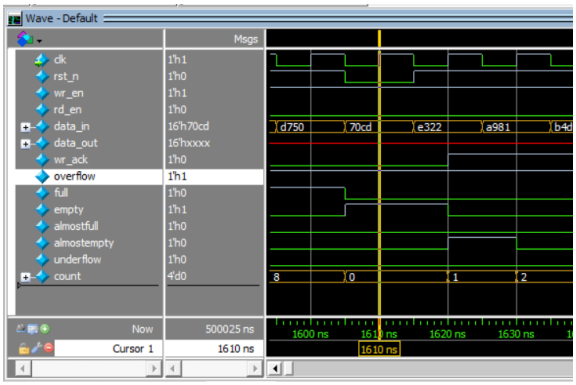
Evidence:



Bug 2: overflow was not reset

The same reset branch as bug 1. The fix is the line overflow <= 0 shown above.

Evidence:



Bug 3: overflow was not cleared by an accepted write

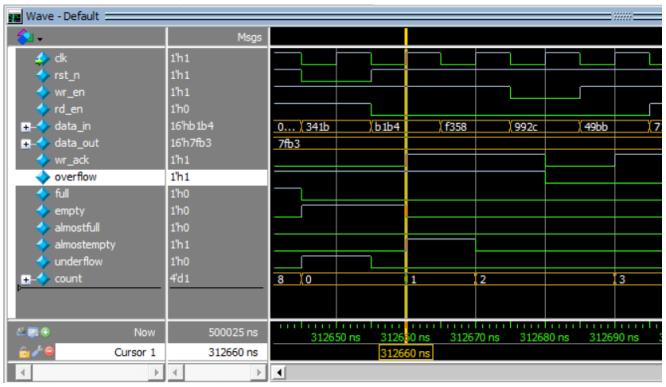
Before:

```
else if (wr_en && count < FIFO_DEPTH) begin
    mem[wr_ptr] <= data_in;
    wr_ack <= 1;
    wr_ptr <= wr_ptr + 1;
end
```

After:

```
else if (wr_en && count < FIFO_DEPTH) begin
    mem[wr_ptr] <= data_in;
    wr_ack <= 1;
    wr_ptr <= wr_ptr + 1;
    overflow <= 0;
end
```

Evidence:



Bug 4: underflow was a combinational assign

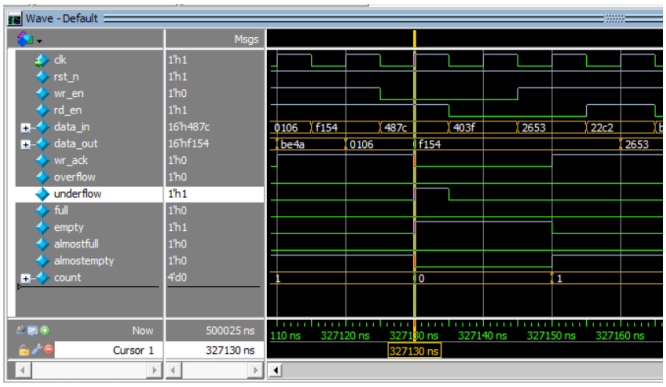
Before:

```
assign underflow = (empty && rd_en)? 1 : 0;
```

After (in the clocked read block, with reset):

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rd_ptr <= 0;
        underflow <= 0;
    end
    else if (rd_en && count != 0) begin
        data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
        underflow <= 0;
    end
    else if (rd_en) begin
        underflow <= 1;
    end
    else begin
        underflow <= 0;
    end
end
```

Evidence:



Bug 5: count was not updated when wr_en and rd_en were both high

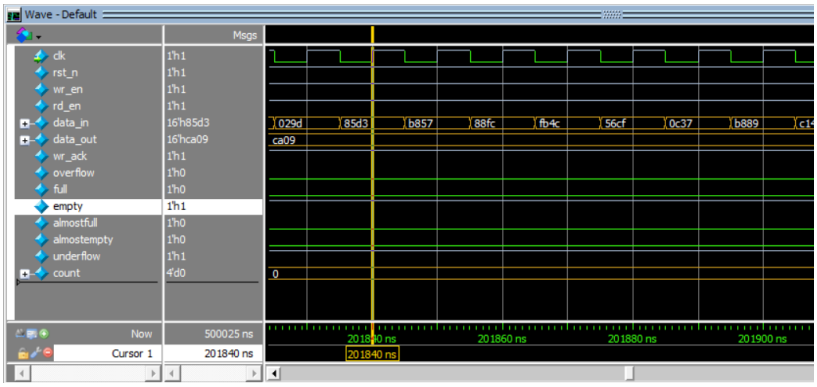
Before:

```
if      ( ({wr_en, rd_en} == 2'b10) && !full)
    count <= count + 1;
else if ( ({wr_en, rd_en} == 2'b01) && !empty)
    count <= count - 1;
```

After:

```
if ({wr_en, rd_en} == 2'b11) begin
    if (empty)      count <= count + 1;   // only the write happens
    else if (full)  count <= count - 1;   // only the read happens
end
else if (wr_en && !full)  count <= count + 1;
else if (rd_en && !empty) count <= count - 1;
```

Evidence:



Bug 6: almostfull compared count with depth - 2

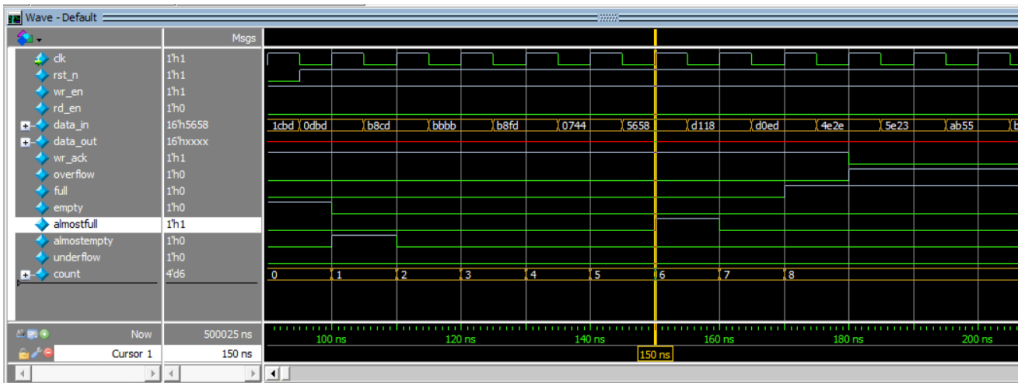
Before:

```
assign almostfull = (count == FIFO_DEPTH-2)? 1 : 0;
```

After:

```
assign almostfull = (count == FIFO_DEPTH-1)? 1 : 0;
```

Evidence:



After the fixes no assertion fails, the scoreboard reports 0 failed transactions, and the code, functional and assertion coverage are all 100% (section 6).